

Math Camp 2026

Algebra and functions • R and Positron

Zhilin Lu

2026-08-13

Algebra and functions

Why we start here

- The first-year sequence does not assume talent. It assumes fluency.
- Fluency means you read $\sum_{i=1}^n (y_i - \bar{y})^2$ without slowing down.
- Every notation gap costs you attention you would rather spend on the argument.
- Four days will not make you a mathematician. It will make the notation stop being noise.

Roadmap for this morning

1. Sets, intervals, notation
2. Summation and product notation
3. Exponents, roots, fractions, order of operations
4. Equations and inequalities
5. Functions: domain, range, composition
6. Polynomials, exponentials, logarithms
7. Where these show up: log-odds, elasticities, log-likelihoods
8. Limits and continuity

A **set** is a collection of distinct objects.

- $A = \{1, 2, 3\}$ — listed
- $B = \{x : x > 0\}$ — defined by a rule, read “the set of x such that $x > 0$ ”
- $x \in A$ means x is an element of A ; $x \notin A$ means it is not

Number systems you will see:

$$\mathbb{N} \subset \mathbb{Z} \subset \mathbb{Q} \subset \mathbb{R}$$

Naturals, integers, rationals, reals.

Set operations

- Union: $A \cup B$ — in A , or B , or both
- Intersection: $A \cap B$ — in both
- Complement: A^c — everything in the universe not in A
- Empty set: $\emptyset$
- Subset: $A \subseteq B$ — every element of A is in B

Notation	Meaning	Set form
(a, b)	open	$\{x : a < x < b\}$
$[a, b]$	closed	$\{x : a \leq x \leq b\}$
$[a, b)$	half-open	$\{x : a \leq x < b\}$
$(-\infty, b]$	unbounded	$\{x : x \leq b\}$

A square bracket includes the endpoint; a parenthesis excludes it. Infinity always gets a parenthesis — it is not a number you can reach.

Summation notation

$$\sum_{i=1}^n x_i = x_1 + x_2 + \cdots + x_n$$

- i is the index, 1 the start, n the stop
- The index name is arbitrary: $\sum_i x_i = \sum_j x_j$

Useful rules:

$$\begin{aligned}\sum_{i=1}^n c x_i &= c \sum_{i=1}^n x_i & \sum_{i=1}^n (x_i + y_i) &= \sum_i x_i + \sum_i y_i \\ \sum_{i=1}^n c &= nc\end{aligned}$$

Summation: what it is not

$$\sum_i x_i y_i \neq \left(\sum_i x_i \right) \left(\sum_i y_i \right)$$

$$\sum_i x_i^2 \neq \left(\sum_i x_i \right)^2$$

The sample mean and variance in one line each:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i \quad s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

$$\prod_{i=1}^n x_i = x_1 \cdot x_2 \cdots x_n$$

- $\prod_i c = c^n$
- Products of many small numbers underflow fast — which is why we take logs
- The likelihood of n independent observations is a product:

$$L(\theta) = \prod_{i=1}^n f(x_i \mid \theta)$$

Hold that thought until the log slide.

$$x^a x^b = x^{a+b} \quad \frac{x^a}{x^b} = x^{a-b} \quad (x^a)^b = x^{ab}$$

$$x^0 = 1 \quad (x \neq 0) \quad x^{-a} = \frac{1}{x^a} \quad (xy)^a = x^a y^a$$

Roots are fractional exponents:

$$\sqrt[n]{x} = x^{1/n} \quad \sqrt{x^2} = |x|$$

The square root returns the non-negative root.

Fractions and order of operations

$$\frac{a}{b} + \frac{c}{d} = \frac{ad + bc}{bd} \quad \frac{a/b}{c/d} = \frac{ad}{bc}$$

Order: parentheses → exponents → multiplication and division (left to right) → addition and subtraction (left to right).

In code this matters as much as on paper:

```
-2^2      # -4, exponent binds tighter than the minus sign
(-2)^2    #  4
1:3 * 2    # 2 4 6
1:(3 * 2)  # 1 2 3 4 5 6
```

$$y = mx + b$$

- m is the slope: change in y per one-unit change in x
- b is the intercept: y when $x = 0$
- Through two points: $m = \frac{y_2 - y_1}{x_2 - x_1}$

Every regression coefficient you report is a slope. When you write “a one-unit increase in x is associated with a $\hat{\beta}$ increase in y ,” this is the sentence you are unpacking.

Quadratic equations

$$ax^2 + bx + c = 0 \implies x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

- Discriminant $b^2 - 4ac$: positive $\rightarrow$ two real roots, zero $\rightarrow$ one, negative $\rightarrow$ none
- Factoring first is usually faster: $x^2 - 5x + 6 = (x - 2)(x - 3)$

Same moves as equations, with one exception:

$$-2x < 6 \implies x > -3$$

Multiplying or dividing by a negative number **flips the inequality**.

- $a < b$ and $c > 0 \implies ac < bc$
- $a < b$ and $c < 0 \implies ac > bc$
- Adding anything to both sides is always safe

$$|x| = \begin{cases} x & x \geq 0 \\ -x & x < 0 \end{cases}$$

Read it as distance from zero, so $|x - a|$ is the distance between x and a .

- $|x| < c \iff -c < x < c$
- $|x| > c \iff x < -c \text{ or } x > c$

The first line is every confidence interval you will ever construct: $|\hat{\theta} - \theta| < 1.96 \cdot SE$.

A **function** f assigns exactly one output to each input.

- **Domain:** the set of legal inputs
- **Range:** the set of outputs actually produced
- Notation: $f : \mathbb{R} \rightarrow \mathbb{R}, f(x) = x^2$

Domains are where the traps are:

Function	Domain
$1/x$	$x \neq 0$
$\sqrt{x}$	$x \geq 0$
$\ln x$	$x > 0$

Composition applies one function to the output of another:

$$(f \circ g)(x) = f(g(x))$$

Order matters. If $f(x) = x^2$ and $g(x) = x + 1$, then $f(g(2)) = 9$ but $g(f(2)) = 5$.

An **inverse** f^{-1} undoes f : $f^{-1}(f(x)) = x$. It exists only when f is one-to-one.

Logistic regression is composition: a linear function, then the logistic function.

$$f(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$$

- **Degree** is the highest power; it caps the number of real roots and turning points
- Degree 1 is a line, degree 2 a parabola
- x^2 terms in a regression are how you let a slope change with x

$$f(x) = a^x, \quad a > 0$$

- Constant *proportional* change per unit of x , not constant absolute change
- Growth if $a > 1$, decay if $0 < a < 1$
- The natural base $e \approx 2.71828$ and $f(x) = e^x$ is its own derivative

Compare: adding 3 every year versus multiplying by 1.03 every year. Political and economic quantities usually do the second.

The logarithm is the inverse of the exponential:

$$y = \log_a x \iff a^y = x$$

$$\log(xy) = \log x + \log y \qquad \log \frac{x}{y} = \log x - \log y$$

$$\log x^k = k \log x$$

- $\ln x$ means $\log_e x$; in $\mathbb{R}$, $\log ()$ is natural log by default
- $\log x$ is defined only for $x > 0$
- Products become sums

Motivating case 1: log-odds

For a probability $p \in (0, 1)$:

$$\text{odds} = \frac{p}{1-p} \quad \text{logit}(p) = \ln \frac{p}{1-p}$$

- p lives in $(0, 1)$; the logit lives in $(-\infty, \infty)$
- That is exactly what lets us put a linear model on the right side:

$$\ln \frac{p_i}{1-p_i} = \beta_0 + \beta_1 x_i$$

Inverting gives the logistic function. Composition, again.

Take logs of both sides:

$$\ln y = \beta_0 + \beta_1 \ln x \implies \beta_1 = \frac{\% \Delta y}{\% \Delta x}$$

Model	Interpretation of β_1
y on x	unit change in y per unit of x
$\ln y$ on x	$\sim 100\beta_1$ % change in y per unit of x
$\ln y$ on $\ln x$	% change in y per % change in x
y on $\ln x$	unit change in y per % change in x

Motivating case 3: log-likelihoods

The likelihood is a product; the log-likelihood is a sum:

$$L(\theta) = \prod_{i=1}^n f(x_i \mid \theta) \implies \ell(\theta) = \sum_{i=1}^n \ln f(x_i \mid \theta)$$

Why we always maximize the log:

- $\ln$ is strictly increasing, so the maximizer is unchanged
- Sums differentiate cleanly; products do not
- Products of 500 numbers below 1 underflow to zero in floating point

Tomorrow morning we maximize this thing.

$$\lim_{x \rightarrow a} f(x) = L$$

“As x gets arbitrarily close to a , $f(x)$ gets arbitrarily close to L .” Note $f(a)$ itself need not exist.

$$\lim_{x \rightarrow 2} \frac{x^2 - 4}{x - 2} = \lim_{x \rightarrow 2} (x + 2) = 4$$

The function is undefined at $x = 2$; the limit is fine. This gap between “value” and “limit” is the whole idea of the derivative.

f is **continuous** at a when three things hold:

1. $f(a)$ exists
2. $\lim_{x \rightarrow a} f(x)$ exists
3. they are equal

Informally: you can draw it without lifting the pen.

- Polynomials, e^x , and $\sin x$ are continuous everywhere
- $1/x$ is not continuous at 0
- Continuity is what lets us take derivatives

You should now be able to:

- Read and manipulate $\sum$ and $\prod$
- Move between exponential and log form without thinking
- State the domain of a function before plotting it
- Explain why we log a likelihood
- Say what a limit is without hand-waving

R and Positron

- One IDE, both languages: R today, Python on Day 4, no context switching
- Built by Posit, the RStudio team — the muscle memory transfers
- The console, editor, environment, and files panes work the way you expect

If anything failed to install, flag it now. Do not spend the afternoon watching someone else's screen.

Two different things, and telling them apart will save you hours later.

- **R** is the language plus a program that understands it. It has almost no interface of its own: it takes instructions and returns results.
- **Positron** is the shell around it — somewhere to write, somewhere to see results, somewhere to see what exists. **It computes nothing.** It hands everything to R.

R is the engine; Positron is the dashboard.

When you say “Positron is broken,” the broken thing is usually R underneath — a package that did not install, an object that does not exist.

The four panes

Pane	What it holds	The catch
Console	runs a line immediately	gone when you quit
Editor	your script	saved to disk
Environment	objects currently in memory	this is “what R knows right now”
Files	the folder R is standing in	check here when a path fails

Work in the **editor**. Send lines to the console with `Cmd/Ctrl + Enter`.

The console is for one-off things: installing a package, glancing at a value.

If it is worth doing twice, it belongs in the editor.

Anything you typed only into the console is gone tomorrow, and it cannot go into a replication package.

Shortcuts worth learning today

Action	Keys
Run the current line	Cmd/Ctrl + Enter
Type <-	Alt + -
Type the pipe \ >	Cmd/Ctrl + Shift + M
Clear the console	Ctrl + L
Previous command	↑ in the console

The last one matters more than it looks. When you mistype a line, do not retype it — press ↑ and fix the one character.

A path is a file's address. There are two kinds.

Absolute — starts at the root of the disk:

```
/Users/zhilin/Documents/mathcamp/data/raw.csv
```

Precise, and true on exactly one computer.

Relative — starts wherever you currently are:

```
data/raw.csv
```

Short, but it needs a “here” to start from.

The working directory is the “here”

R is always standing in some folder. Every relative path is measured from it.

```
getwd()
```

```
[1] "/Users/zhilin/mathcamp"
```

Three lines that diagnose almost every “it can’t find my file”:

```
getwd()           # where am I standing
list.files("data/raw") # what is in that folder
file.exists("data/raw/trade-panel.csv") # is this exact file there
```

Projects set the “here” for you

Open the **project folder** in Positron, not a lone .R file. The working directory becomes that folder automatically.

Now relative paths work — on your machine, on your coauthor's, on the reviewer's.

```
here::here()  
here::here("data", "raw", "trade-panel.csv")
```

```
[1] "/Users/zhilin/mathcamp"
```

```
[1] "/Users/zhilin/mathcamp/data/raw/trade-panel.csv"
```

`here()` also handles the Windows-vs-Mac slash problem, which hand-written paths do not.

Never use `setwd( )`

```
setwd("/Users/zhilin/Documents/mathcamp") # do not do this
```

It hard-codes **your** machine into the script. It will fail for your coauthor, for the journal's replication check, and for you in three years on a new laptop.

If your script contains your own username, it is already broken. Nobody has noticed yet.

A folder that will not embarrass you

```
math-camp/  
├─ math-camp.Rproj  
├─ data/  
│   ├─ raw/          <- read-only. never edited by hand.  
│   └─ clean/        <- produced by scripts. deletable.  
├─ R/  
│   ├─ 01-clean.R  
│   └─ 02-analyze.R  
└─ output/
```

The numbers in the filenames say what order to run things in.

The test: delete data/clean/ entirely and re-run. Everything should come back. If it does not, some step happened by hand and was never written down.

`x <- 5` means: make a box in memory, call it x, put 5 in it.

```
x <- 5  
x  
x * 3
```

```
[1] 5
```

```
[1] 15
```

Notice that `x <- 5` printed **nothing**. Assignment is silent. To see a value, type the name on its own line.

`<-` is the convention; `Alt + -` types it. Names are case sensitive: `X` and `x` are different objects.

Values are not formulas

```
y <- x + 2  
y  
x <- 10  
y
```

```
[1] 7
```

```
[1] 7
```

y is still 7. It stored **the value computed at the time**, not a rule that keeps tracking x.

This is the opposite of a spreadsheet formula, and it surprises people.

```
ls()
```

```
[1] "x" "y"
```

That is exactly what the Environment pane is showing you.

Arithmetic

```
7 / 2  
7 %/% 2  
7 %% 2  
2^10
```

```
[1] 3.5
```

```
[1] 3
```

```
[1] 1
```

```
[1] 1024
```

%/% is integer division, %% is the remainder.

Precedence bites here:

```
-2^2  
(-2)^2
```

```
[1] -4
```

```
[1] 4
```

The exponent binds tighter than the minus sign.

Numbers are not exact

```
0.1 + 0.2 == 0.3
```

```
[1] FALSE
```

Not a bug in R. 0.1 cannot be written exactly in binary, the same way 1/3 cannot be written exactly in decimal. Every language does this.

```
all.equal(0.1 + 0.2, 0.3)
```

```
[1] TRUE
```

Never test computed numbers with `==`. You will meet this again the first time you write `if (p == 0.05)`.

What is that [1]?

It is the position of the first element on that line — because **everything in R is a vector**, even a single number.

```
seq(0, 30, by = 3)
```

```
[1]  0  3  6  9 12 15 18 21 24 27 30
```

Longer output wraps, and each new line is labelled with the index it starts at. That is all it means.

```
class(5)
class("Cornell")
class(TRUE)
class(3L)
```

```
[1] "numeric"
```

```
[1] "character"
```

```
[1] "logical"
```

```
[1] "integer"
```

Text needs quotes. Without them, R goes looking for an object with that name.

Types decide what is allowed

```
"5" + 5
```

Error in `"5" + 5` : non-numeric argument to binary operator

Read it literally: a non-numeric thing showed up where arithmetic expected a number.

```
as.numeric("5") + 5  
as.numeric("abc")
```

```
[1] 10
```

```
[1] NA
```

Warning message:

NAs introduced by coercion

A **warning** is not an error. The code finished; something in it could not be read. Warnings are usually good news — they tell you how much went wrong.

Logicals are secretly numbers

```
TRUE + TRUE
```

```
mean(c(TRUE, FALSE, TRUE, TRUE))
```

```
[1] 2
```

```
[1] 0.75
```

TRUE counts as 1 and FALSE as 0, so **the mean of a logical vector is a proportion**.

You will use this constantly: `mean(df$year > 2000)` is the share of observations after 2000.

Factors

A factor is a categorical variable: party, region, regime type. It looks like text and is stored as codes plus a table of labels.

```
party <- factor(c("D", "R", "D", "I", "R"), levels = c("D", "R", "I"))  
party  
table(party)
```

```
[1] D R D I R  
Levels: D R I
```

```
party  
D R I  
2 2 1
```

That `Levels:` line is how you recognise a factor. The level order — not alphabetical order — is what sets the reference category in a regression and the order of bars in a plot.

The factor trap

Numbers that were stored as a factor by accident:

```
scores <- factor(c("10", "20", "30"))  
as.numeric(scores)
```

```
[1] 1 2 3
```

Those are the **codes**, not the values. The correct route:

```
as.numeric(as.character(scores))
```

```
[1] 10 20 30
```

Nothing errored. Code that runs and is wrong is far more dangerous than code that stops.

Vectors

A vector is a run of values of one type. **A single number is a vector of length 1** — that is where the [1] comes from.

```
v <- c(3, 1, 4, 1, 5)
length(v)
```

```
[1] 5
```

One type only. Mix them and everything is silently converted:

```
c(1, "a", TRUE)
```

```
[1] "1"      "a"      "TRUE"
```

One stray "n/a" in a column of numbers turns the whole column into text. Tomorrow afternoon you will meet exactly this.

Building vectors

```
1:5  
seq(0, 10, by = 2)  
rep(0, 3)
```

```
[1] 1 2 3 4 5
```

```
[1] 0 2 4 6 8 10
```

```
[1] 0 0 0
```

Indexing starts at 1

```
v[1]  
v[c(1, 3)]  
v[2:4]
```

```
[1] 3  
[1] 3 4  
[1] 1 4 1
```

Out of range gives NA rather than an error:

```
v[10]
```

```
[1] NA
```

A minus sign means “drop”

```
v[-1]  
v[-c(1, 2)]
```

```
[1] 1 4 1 5
```

```
[1] 4 1 5
```

Not “count from the end.”

Vectorized arithmetic

```
v * 2  
v^2  
v + c(10, 20, 30, 40, 50)
```

```
[1] 6 2 8 2 10
```

```
[1] 9 1 16 1 25
```

```
[1] 13 21 34 41 55
```

No loop. Operations apply element by element.

R was built by statisticians, and in statistics you never handle one number — you handle a column of observations. So “do this to every observation” is the default behaviour, not something you have to ask for.

Writing a for loop where a vectorized call would do is the clearest beginner tell, and on real data it is slower by orders of magnitude.

Short vectors repeat to match long ones.

```
v + c(10, 20)
```

```
[1] 13 21 14 21 15
```

Warning message:

longer object length is not a multiple of shorter object length

This is why `v * 2` works at all — the 2 is recycled five times.

It warns when the lengths do not divide evenly. When they **do** divide evenly it happens silently, and that is the case that bites.

Logical subsetting

Three steps. Run them separately.

```
v > 2  
sum(v > 2)  
v[v > 2]
```

```
[1] TRUE FALSE TRUE FALSE TRUE
```

```
[1] 3
```

```
[1] 3 4 5
```

1. The test gives a vector of TRUE/FALSE
2. TRUE is 1, so the sum counts how many passed
3. The brackets keep the positions that were TRUE

Tomorrow's filter() is this exact idea, applied to whole rows.

Missing values

```
w <- c(3, 1, NA, 5)
mean(w)
mean(w, na.rm = TRUE)
```

```
[1] NA
```

```
[1] 3
```

NA is contagious **by design**: the mean of three known numbers and one unknown really is unknown. R refuses to guess for you.

```
is.na(w)
w == NA
```

```
[1] FALSE FALSE TRUE FALSE
```

```
[1] NA NA NA NA
```

`is.na()` is the test. `w == NA` asks whether an unknown equals an unknown — which is itself unknown.

A table. Underneath, a set of equal-length vectors stacked side by side.

```
df <- data.frame(  
  country = c("USA", "JPN", "NLD", "BRA"),  
  region  = c("Americas", "Asia", "Europe", "Americas"),  
  exports = c(1.6, 0.7, 0.6, 0.3)  
)  
df
```

	country	region	exports
1	USA	Americas	1.6
2	JPN	Asia	0.7
3	NLD	Europe	0.6
4	BRA	Americas	0.3

Columns may differ in type. Rows may not, and every column must be the same length.

Look before you touch

```
str(df)
```

```
'data.frame':  4 obs. of  3 variables:  
 $ country: chr  "USA" "JPN" "NLD" "BRA"  
 $ region : chr  "Americas" "Asia" "Europe" "Americas"  
 $ exports: num  1.6 0.7 0.6 0.3
```

How many rows, how many columns, what type each one is — in one line. The first thing to run on any new dataset.

```
nrow(df); ncol(df); names(df)
```

Columns are vectors

```
df$exports  
mean(df$exports)
```

```
[1] 1.6 0.7 0.6 0.3
```

```
[1] 0.8
```

\$ hands you an ordinary vector, so everything from the last twenty minutes applies unchanged.

Making a new column uses the same \$:

```
df$exports_pct <- df$exports * 100
```

Brackets: [rows, columns]

The comma is the whole trick. Before it, rows. After it, columns. Blank means “all”.

```
df[1, ]  
df[, "exports"]  
df[2, 3]
```

	country	region	exports
1	USA	Americas	1.6

```
[1] 1.6 0.7 0.6 0.3
```

```
[1] 0.7
```

Selecting rows the hard way

```
df[df$region == "Americas", ]
```

	country	region	exports
1	USA	Americas	1.6
4	BRA	Americas	0.3

It works, and it is the logical subsetting you already know. But `df$` appears twice, and with a longer name it gets ugly fast.

Hold that annoyance. Tomorrow afternoon `filter()` removes it.

```
library(tibble)
as_tibble(df)
```

```
# A tibble: 4 x 3
  country region  exports
  <chr>   <chr>    <dbl>
1 USA     Americas  1.6
2 JPN     Asia      0.7
3 NLD     Europe    0.6
4 BRA     Americas  0.3
```

A data frame with better manners:

- prints its dimensions and **the type of every column**
- stops at 10 rows instead of flooding the console
- never silently converts text to factors

It is still a data frame, so old code keeps working. We use tibbles from tomorrow on.

Writing a function

```
standardize <- function(x) {  
  (x - mean(x)) / sd(x)  
}  
standardize(c(3, 1, 4, 1, 5))
```

```
[1] 0.1118034 -1.0062306 0.6708204 -1.0062306 1.2298374
```

- `function(x)` — `x` is the **argument**, a placeholder for whatever gets passed in
- the braces hold the body
- the last expression is returned; `return()` is optional

Type the name without parentheses to see the definition of any function.

Arguments and defaults

```
gdp_per_capita <- function(gdp, pop, units = 1000) {  
  gdp / pop * units  
}  
gdp_per_capita(923.7, 162)  
gdp_per_capita(923.7, 162, units = 1)  
gdp_per_capita(pop = 162, gdp = 923.7)
```

```
[1] 5701.852
```

```
[1] 5.701852
```

```
[1] 5701.852
```

Defaults are used unless you override them. Positional arguments must be in order; **named arguments can be in any order**, and they read better.

What happens inside stays inside

```
z <- 999
f <- function() {
  z <- 1
  z
}
f()
z
```

```
[1] 1
```

```
[1] 999
```

A feature, not a limitation: calling someone else's function cannot quietly wreck your own objects.

The rule of three. Copying the same code a third time means it should be a function.

?mean

Section	What it is
Description	one sentence
Usage	how to call it, and the defaults
Arguments	what each one means
Value	what comes back
Examples	runnable code

Read it backwards. Scroll to Examples, run one, then go up to Arguments when something is unclear. Reading top to bottom is how beginners give up.

```
args(round)
example(sd)
vignette(package = "dplyr")  # long-form guides, much friendlier
```

```
install.packages("tidyverse")  # download it. once per computer.  
library(tidyverse)            # open it. every session.
```

Buying a book and putting it on the shelf, versus taking it down and opening it.

Two rules:

- `install.packages()` lives in the **console**, never in a script — otherwise everyone who runs your code re-downloads everything
- all `library()` calls go at the **top** of the script, together, so a reader knows immediately what they need

Read the startup message

```
library(dplyr)
```

The following objects are masked from 'package:stats':

```
filter, lag
```

Masking means dplyr defined its own `filter()` and it now sits in front of the one R came with.

```
search()[1:3]
```

```
[1] ".GlobalEnv"      "package:dplyr" "package:stats"
```

R searches that list in order, so the most recently loaded package wins. To be explicit, name the package:

```
dplyr::filter(df, exports > 0.5)
```

When a function behaves strangely, ask what masked it.

Errors you will meet this week

```
mean(vv)
```

Error: object 'vv' not found

A typo, or you skipped the line that creates it. Check the Environment pane.

```
meen(v)
```

Error: could not find function "meen"

Typo in the function name — or the package is not loaded.

there is no package called 'X'

Not installed. And if the installer asks whether to **install from sources**, say no.

Say the message out loud before doing anything else. It is usually literal.

Before tomorrow

- Confirm `library(tidyverse)` loads without errors — **tomorrow afternoon depends on it**
- Skim R for Data Science, chapter 1
- Finish the exercise

Tomorrow: calculus at 9:00, linear algebra at 11:00, data wrangling after lunch.